

GRAPH-GUIDED LONG-HORIZON MPC WITH WORLD MODELS

Anonymous authors

Paper under double-blind review

ABSTRACT

The cross-entropy method (CEM) planner used with LeWorldModel (LeWM) ranks candidate action sequences by the Euclidean distance between a predicted terminal latent and a goal latent. While this objective succeeds on short-horizon goals, its performance degrades sharply as goals become more temporally distant or come from a different episode. On Push-T, the planner reaches same-episode goals 25, 50 and 100 steps ahead, and cross-episode goals, with success rates of 91/41/13/14%, respectively. We address this degradation at long horizons and across episodes without retraining the world model or learning a subgoal generator. We learn a temporal distance representation over frozen latents, build a graph of the offline dataset, and select a subgoal along the graph’s shortest path. A reachability critic scores predicted endpoints by expected time to goal, and latent L2 is used for the final horizon. Each component addresses a distinct limitation: the subgoal provides *direction*, the critic accounts for *reachability*, and the switch to L2 restores near-goal *resolution*. The graph finds a route for every evaluated cross-episode pair, but reaching the goal still requires the critic. Across goals at offsets 50 and 100, and on cross-episode goals, our method raises success from 41/13/14% to 70/46/42% on Push-T, from 91/83/29% to 98/98/66% on Reacher, and from 34/36/16% to 50/50/28% on Cube. At the 25-step horizon, our method trails L2 by 2.8 points on Push-T (88.4 vs 91.2) and 1.2 points on Reacher (76.0 vs 77.2), but leads by 6.0 points on Cube (62.0 vs 56.0).

1 INTRODUCTION

Key points this section must land.

- The problem is the *objective*, not the model: terminal latent L2 stops ranking candidates past ~ 50 steps and plateaus on cross-episode goals at 14% even with a $5\times$ larger budget.
- Existing fixes retrain the encoder, add predictors, or learn generators. We keep the world model and its planner frozen and use the dataset itself as a graph.
- Three mechanisms, three failures: direction (graph subgoal), reachability (hitting-time critic), resolution (final switch). Neither subgoal nor critic alone moves cross-episode.
- Cross-episode goal reaching is the headline evaluation; same-episode long horizons are supporting evidence.
- Contributions list, four items, each tied to a table or figure.

Joint-embedding world models trained from pixels (Maes et al., 2026; Zhou et al., 2024; Sobal et al., 2025) plan by search: they sample candidate action sequences, roll each through a learned latent predictor, and select the sequence whose predicted terminal latent lies closest to the goal latent in Euclidean (L2) distance – the default objective we call L2 throughout. On the horizons these models are typically evaluated on, where the goal is a frame from the same trajectory a fixed number of steps ahead, this objective performs well. Its performance degrades sharply, however, as the goal becomes more temporally distant. With the frozen LeWorldModel (LeWM) checkpoint and its published cross-entropy-method (CEM) planner, Push-T success is 91% for goals 25 steps ahead, 41% at 50 steps, 13% at 100 steps, and 14% for a goal drawn from a different episode. Additional budget barely helps: L2 reaches 12.0% of cross-episode goals within a 100-step budget and it only increases to 14.4% with the full 250-step budget (table 8).

The failure lies in the objective rather than the model. A privileged audit shows that terminal-latent distance ranks candidate endpoints well at 25 steps (Spearman correlation of 0.72 with a recovery-time oracle) but poorly at 50 steps (0.36); a single-pass audit further shows that the predicted terminal cost is optimistic on 88% of tasks and barely separates successes from failures (section 3). The planner is not wrong about where the goal is; it is wrong about which of its candidates can still reach it.

A substantial body of recent work targets the same failure in the same frozen-model setting, almost uniformly by learning something new on top of it: a subgoal generator and action prior (Cheng et al., 2026), a macro-action space and high-level predictor (Caselli et al., 2026), world models operating at several time scales (Zhang et al., 2026), an action-free latent planner (Masip et al., 2026), or a retrained encoder whose geometry directly encodes temporal cost (Bai & Xiong, 2026; Li et al., 2026b;a). A separate, older line of work in offline goal-conditioned reinforcement learning addresses long horizons by searching over the data directly: it builds a graph over the offline dataset and lets shortest paths select subgoals (Savinov et al., 2018; Eysenbach et al., 2019; Emmons et al., 2020; Baek et al., 2025; Opryshko et al., 2025; ALPS authors, 2026), handing the resulting subgoal to a learned goal-conditioned policy. To our knowledge, no prior method places the dataset graph inside a pixel world model’s planner itself, and Caselli et al. (2026) identify a plausible reason: their hierarchical planner fails whenever its generated subgoals cease to be “data-supported, executable, and matched to the temporal scale.” A subgoal that is itself a dataset frame satisfies all three by construction.

This paper takes that approach. We learn a temporal distance representation (TDR) on the frozen LeWM latents (Park et al., 2024; Baek et al., 2025), build a graph over the full offline dataset in that space, and replace LeWM’s terminal cost with the distance to a subgoal: the graph node one planning horizon along the shortest path toward the goal. This alone repairs the mid-range regime (Push-T offset 50: 42.0 $\rightarrow$ 51.6%, table 2) but does nothing for cross-episode goals: by the graph’s own geodesic, every cross-episode pair is 45–120 steps apart, well inside the planning budget, yet the planner does not execute the route. The missing component is reachability. We add a critic $V(z, z_g, h)$, the probability that the goal is reachable from latent z within h steps, and score each predicted endpoint by a budget-capped *expected hitting time* under the steps remaining after the plan being scored. With this term, and with the final horizon handed back to LeWM’s own objective once the goal is one horizon away, a single configuration matches or exceeds every alternative we tested at every horizon of 50 steps or more, on same-episode and cross-episode goals alike, across three environments (table 1). Averaged over offsets 50 and 100 and cross-episode goals, our method raises success from 41/13/14% to 70/46/42% on Push-T, from 91/83/29% to 98/98/66% on Reacher, and from 34/36/16% to 50/50/28% on Cube. At the 25-step horizon our method trails L2 by 2.8 points on Push-T (88.4 vs 91.2) and 1.2 points on Reacher (76.0 vs 77.2), within CEM sampling noise (section C), while leading by 6.0 points on Cube (62.0 vs 56.0).

Our contributions are:

1. A planner-side method for long-horizon and cross-episode goal reaching with a frozen pixel world model, requiring no new generator, policy, or predictor: a TDR on frozen latents, a dataset graph, a hitting-time critic, and an unmodified CEM (section 4).
2. A budget-clocked expected-hitting-time cost that replaces the standard $-\log V$ feasibility term, removes its dependence on a weighting hyperparameter, and preserves a progress gradient among feasible candidates (sections 4.3 and 5.3).
3. The first cross-episode goal-reaching evaluation for latent MPC on Push-T, Reacher, and Cube, using fixed task pools shared by every method and seed (section 5.1 and table 1).
4. A mechanistic account, supported by per-pair diagnostics, of which failure mode each component addresses: direction, reachability, and resolution (section 5.4).

2 RELATED WORK

Key points this section must land.

- Group by what is learned, not by year: (i) planner-side costs on a frozen model, (ii) hierarchical planning with learned generators/predictors, (iii) graph search over offline data with learned policies, (iv) value functions inside MPC.
- For each group say in one clause what we take and in one clause what we do not.
- Be explicit that SAGE reports higher same-episode numbers at $H = 150$ and cannot stitch by construction; do not hide it, it is our argument.

Planning with latent world models. PlaNet and Dreamer (Hafner et al., 2019; 2023) plan or learn policies in a reconstruction-based latent; DINO-WM (Zhou et al., 2024), PLDM (Sobal et al., 2025), V-JEPA 2-AC (Assran et al., 2025) and LeWM (Maes et al., 2026) plan with zero-shot CEM in a joint-embedding latent, scoring candidates by terminal latent distance. Terver et al. (2025) and stable-worldmodel authors (2026) study which design choices matter; the planning horizon is a known weak point of all of them.

Repairing the cost on a frozen model. Trajectory reachability metrics (Li et al., 2026a) learn a pairwise temporal cost from logged trajectories and use it as the terminal ranking; temporal-distance JEPA (Bai & Xiong, 2026), RC-aux (Li et al., 2026b), DA-LeWM (Wang et al., 2026) and SCALE (Hu et al., 2026) push similar signals into the encoder at training time, and Destrade et al. (2026); Temporal straightening authors (2026) shape the geometry toward a value or a straight path. Our TDR is in the same family (an expectile temporal distance on frozen latents) but we do not use it as the cost: we use it to build a graph and pick a data-supported target, and we show the metric alone is not what helps (section 5.3). RC-aux’s budget-conditioned reachability is the closest relative of our critic; ours is a separate network with an explicit plan clock and never touches the encoder.

Hierarchical planning with world models. HWM (Zhang et al., 2026) learns world models at several temporal scales and uses long-horizon predictions as subgoals; SAGE (Cheng et al., 2026) learns a subgoal generator and a subgoal-conditioned action prior that CEM refines; FF-JEPA (Masip et al., 2026) learns an action-free latent planner; Hi-LeWM (Caselli et al., 2026) learns macro-actions and a high-level predictor and shows the naive version loses to flat planning because CEM exploits the learned subgoal space. Each of these trains a generative component whose training pairs are same-trajectory windows, so their subgoals cannot lie on a route that no single trajectory took. Our subgoals are dataset frames on a shortest path over the whole dataset, and the cross-episode protocol is where that difference shows. SAGE reports stronger same-episode numbers than ours at long offsets (64.7% at $H = 150$ on Push-T with the same frozen model); we discuss this in section 6.

Graph search over offline data. SPTM (Savinov et al., 2018), SoRB (Eysenbach et al., 2019) and SGM (Emmons et al., 2020) build graphs over replay buffers with a learned distance and plan waypoints for a goal-conditioned policy; GAS (Baek et al., 2025) clusters a temporal distance representation (Park et al., 2024; Wang et al., 2023) into nodes with a temporal-efficiency filter and searches it for subgoals; TTGS (Opryshko et al., 2025) and ALPS (ALPS authors, 2026) do the same at test time for frozen OGBench policies (Park et al., 2025). All of them execute subgoals with a learned policy (or a state-space model with a behaviour prior). We take GAS’s graph construction essentially unchanged and give the subgoal to a pixel world model’s CEM instead; an earlier RL version of GAS on the same latent found the low-level policy, not the graph, to be the win, which is why the executor matters and why we test it separately.

Value functions inside MPC. POLO (Lowrey et al., 2019), TD-MPC (Hansen et al., 2022; 2024) and Director (Hafner et al., 2022) put a learned value at the end of a short model rollout; goal-conditioned terminal values have been used for multi-task MPC (GC terminal value authors, 2024). Those values are trained with rewards or with the model. Our critic is a reachability classifier trained from hindsight and cross-episode negatives on the offline data, and its contribution is a budget-aware hitting time rather than a value.

3 THE PROBLEM: LATENT MPC BEYOND ONE HORIZON

Key points this section must land.

- Define the planner exactly once (300 samples, 30 iterations, 30 elites, 5 blocks of 5 actions, open-loop 25 steps, replan). Everything downstream changes only the criterion.
- Two audits, one figure: (a) oracle Spearman of terminal L2 vs recovery time drops $0.72 \rightarrow 0.36$ from offset 25 to 50; (b) single-pass predicted vs realized terminal cost: optimistic on 88.5% of tasks, Spearman 0.30, medians 2.2 vs 2.3 (pred) against 11 vs 30 (real).
- Cross-episode is budget-limited, not budget-invariant: L2 reaches 6.0% within a 50-step budget, 12.0% within 100, and only 14.4% within the full 250-step budget (table 8).

Frozen world model and planner. LeWM (Maes et al., 2026) consists of a ViT encoder E mapping a 224×224 frame to a 192-d CLS latent z and a predictor P that maps three past latents and three action blocks to the next latent, where an action block is five consecutive environment actions (frame-skip 5). Planning is CEM over $K = 5$ blocks (25 environment steps): 300 candidates, 30 iterations, 30 elites, each candidate scored by the terminal cost $c_{L2} = \|\hat{z}_K - z_g\|^2$ between the predicted block- K latent and the encoded goal frame. The optimum is executed open loop for 25 steps and the planner is called again. We keep the encoder, the predictor and this planner fixed throughout and change only the function that scores $(z_0, \hat{z}_{1..K}, z_g)$.

Where it fails. Table 1 (rows “L2”) gives the picture on three environments: near-perfect one horizon out, a steep fall past two, and a floor for goals from another episode. Two audits locate the failure in the ranking. First, against a privileged recovery-time oracle (the number of steps a strong planner with state access needs from a candidate endpoint), c_{L2} orders endpoints with Spearman 0.72 at offset 25 and 0.36 at offset 50. Second, executing the single 25-step plan of 200 offset-25 tasks and re-encoding the true final frame, the predicted cost under-estimates the realized latent distance on 88.5% of tasks, correlates with it at only $\rho = 0.30$, and barely separates successes from failures (median 2.17 vs 2.31) while the realized distance does (10.9 vs 30.0). The planner optimises a quantity that is optimistic and, past one horizon, uninformative about which endpoints can still reach the goal. On the fixed `task200u` pool, cross-episode success accrues most of its gain early and then saturates: L2 reaches 6.0% within a 50-step budget, 12.0% within 100, and only 14.4% within the full 250-step budget, with the hazard rate for each further 50-step window falling below 1.5% past step 100 (table 8). More budget buys diminishing, not zero, returns.

4 METHOD

Key points this section must land.

- One equation per component. The reader should be able to reimplement from the section.
- Say what is learned (TDR: 3×512 MLP; critic: 5-member MLP ensemble) and what is not (encoder, predictor, CEM, graph is built not trained).
- State the privileged-label dependency of the critic here, in the open.
- The plan clock: h_{rem} is the budget left *after* the plan being scored. This is the single most important implementation detail.
- Only two scalars are environment specific (H_{TD} , λ) and both are read off the TDR’s own step-gap calibration; $\beta = 1$ everywhere.

Two failure modes, two different fixes. Section 3’s 91/41/13/14% success and $0.72 \rightarrow 0.36$ oracle Spearman collapse conflate two separate problems. The first is procedural: committing to an open-loop rollout wider than one horizon compounds predictor error and costs the L2 baseline 6–10 points, growing with commit width (section F); every result in this paper already replans every 25 steps, so this is folded into the baseline rather than something the method fixes. The second is what remains after that fix, and is what the rest of this section targets: even replanning every 25 steps, terminal z -space L2 aimed at the true goal still collapses with horizon, because past $H \approx 50$ the true start-goal latent distance (19.4–19.6) is statistically indistinguishable from two unrelated frames (19.6, section F) – the cost has nothing left to discriminate on – while at $H = 25$ it remains clearly separated (17.1). Section G gives the mechanism: the latent cloud concentrates on a thin

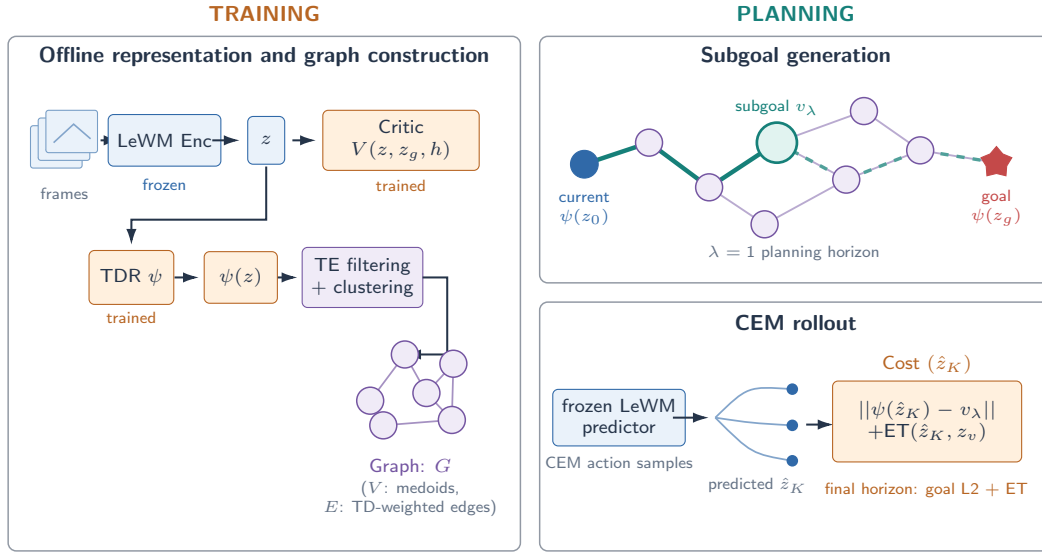


Figure 1: Graph-guided latent MPC. Training (left): frozen LeWM maps frames to z ; the learned TDR maps z to $\psi(z)$ for filtering and clustering into graph G , while a separate reachability critic is trained. Planning (right): graph search selects a subgoal one horizon ahead, and frozen LeWM/CEM scores rollouts by subgoal distance and budget-clocked expected hitting time. Near the goal, goal latent L2 replaces subgoal distance.

isotropic shell, so a straight line between two far-apart points on it passes through the shell’s near-empty interior, exactly what a terminal-L2 CEM plan implicitly draws. The method below keeps raw L2 wherever it is still informative – within one horizon, via the final switch of section 4.2 – and replaces it beyond that with quantities built to stay informative at range: TDR distance in place of raw position (section 4.1), a graph that follows the shell instead of cutting across it (sections 4.1 and 4.2), and a critic that estimates reachability directly rather than through any latent distance (section 4.3).

4.1 TEMPORAL DISTANCE REPRESENTATION AND DATASET GRAPH

We encode every frame of the offline dataset once with the frozen encoder and train a small MLP $\psi : \mathbb{R}^{192} \rightarrow \mathbb{R}^{32}$ so that $\|\psi(z) - \psi(z_g)\|$ approximates the minimum number of steps from z to z_g , using the expectile temporal-difference objective of HILP and GAS (Park et al., 2024; Baek et al., 2025) (expectile 0.99, $\gamma = 0.99$, goals relabelled from the same trajectory with probability 0.625 and uniformly otherwise). We calibrate what one TDR unit means by the median TDR distance between frames k steps apart, u_k , for $k \in \{1, \dots, 50\}$; this table is the only place environment scale enters.

The graph follows GAS: frames whose temporal efficiency (the ratio of TDR progress to steps taken) falls below $\theta_{TE} = 0.9$ are dropped, the rest are clustered sequentially in TDR space at radius $H_{TD}/2$, and nodes within H_{TD} of each other are joined by edges weighted by TDR distance. H_{TD} is u_{12} , the TDR distance of a 12-step gap. Each node stores its medoid frame’s latent. At planning time the goal latent is attached as a temporary node linked to every node within $\max(H_{TD}, 1.2 d_{\min})$ and one Dijkstra pass gives $D_g[v]$, the cost-to-go from every node. On Push-T (18,685 expert episodes, 2.34M frames) the graph has 20.5k nodes; Reacher (10k episodes, 2.01M frames) needs 535; [TODO: Cube]. Building it is a one-off cost of [TODO: x] minutes on one GPU.

4.2 SUBGOAL ONE HORIZON AHEAD, AND THE FINAL SWITCH

At each planning call with current latent z_0 , we select the start node $v^* = \arg \min_{v: \|\psi(z_0) - c_v\| \leq H_{TD}} D_g[v] + \|\psi(z_0) - c_v\|$ and walk the shortest path from v^* toward

the goal until the cumulative TDR distance from z_0 reaches the lookahead $\lambda = u_{25}$, the calibrated distance of one planning horizon. The node reached, v_λ , is the subgoal and the criterion becomes

$$c_{\text{sub}}(\hat{z}_K) = \|\psi(\hat{z}_K) - c_{v_\lambda}\|. \quad (1)$$

Placing the subgoal exactly one horizon out matters: shorter lookaheads leave progress on the table and waypoint tracking over the intermediate predicted latents is worse than the terminal cost alone (section 5.3), because those latents are the predictor’s least reliable outputs. Once the goal itself is within one horizon, $\|\psi(z_0) - \psi(z_g)\| \leq \lambda$, the criterion reverts to LeWM’s own c_{L2} against the true goal frame. This *final switch* is what lets the method reduce to the baseline wherever the baseline is best: a cluster medoid has a radius comparable to the success tolerance, and the TDR norm has less resolution at the goal than the latent L2 – which at that range is itself still discriminative rather than saturated (section F: true start-goal distance 17.1 at $H = 25$, clear of the 19.6 random-pair floor, already 19.4 by $H = 50$).

4.3 BUDGET-CLOCKED EXPECTED HITTING TIME

A reachability critic $V(z, z_g, h) \in [0, 1]$ estimates the probability that the goal can be reached from latent z within h environment steps. It is a five-member MLP ensemble on (z, z_g, h) with h on a grid of multiples of 5 up to $h_{\text{max}} = 50$, trained on the offline dataset with (i) hindsight labels $\mathbb{1}[h \geq \tau]$ where τ is the first step at which the logged trajectory satisfies the environment’s success predicate toward a later frame, (ii) cross-episode negatives whose object or joint configuration differs by more than the data ever moves in 50 steps, (iii) predictor-imagined rollouts under logged actions, (iv) a Bellman backup through the frozen predictor with an EMA target aggregated pessimistically, and (v) a hinge enforcing monotonicity in h . The labels in (i) and the gate in (ii) evaluate the success predicate on the dataset’s state column; the critic’s inputs are latents only, but its training is not pixel-only, and we return to this in section 6.

Rather than the usual $-\log V$ penalty, we score an endpoint by how long the goal is expected to take from it, capped at the budget that will remain once the plan being scored has been executed:

$$ET(\hat{z}_K) = 5 \sum_{h \in \{0, 5, \dots, h_{\text{rem}}\}} (1 - V(\hat{z}_K, z_g, h)), \quad h_{\text{rem}} = \min(h_{\text{max}}, B - t - 5K), \quad (2)$$

where B is the episode budget and t the current step. A *plan clock* supplies t : every CEM call advances all live environments by one horizon, so the critic always answers “reachable in the steps that are left” rather than at a fixed horizon. Feasible endpoints are ordered by how soon they arrive; infeasible ones sit at the cap. The final criterion standardises both terms by median and interquartile range over the 300 CEM candidates of each environment (denoted $\bar{\cdot}$) and adds them,

$$c = \widetilde{c_{\text{sub}}} + \beta \widetilde{ET}, \quad \beta = 1, \quad (3)$$

with c_{L2} in place of c_{sub} in the final phase. Algorithm 1 summarises one planning call.

What is learned, what is not. The TDR (50k steps, batch 1024) and the critic (60k steps) are the only trained components; together they are under [TODO: x]M parameters and take [TODO: x] GPU-hours per environment. The encoder, predictor, CEM hyperparameters and the graph-construction rules are LeWM’s and GAS’s, unchanged. Two scalars are environment specific, $H_{\text{TD}} = u_{12}$ and $\lambda = u_{25}$, and both are read off the TDR calibration table rather than tuned; $\beta = 1$, $\theta_{\text{TE}} = 0.9$ and $h_{\text{max}} = 50$ are shared by every environment and protocol.

Algorithm 1 One planning call of graph-assisted latent MPC

Require: frozen E, P ; CEM; TDR ψ ; graph (V_G, c_v, D_g) ; critic V ; budget B ; step t

- 1: $z_0 \leftarrow E(o_t)$; $h_{\text{rem}} \leftarrow \min(h_{\text{max}}, B - t - 5K)$
- 2: **if** $\|\psi(z_0) - \psi(z_g)\| \leq \lambda$ **then**
- 3: target $\leftarrow$ goal; base cost $\leftarrow c_{L2}$
- 4: **else**
- 5: $v^* \leftarrow \arg \min_{v: \|\psi(z_0) - c_v\| \leq H_{\text{TDR}}} D_g[v] + \|\psi(z_0) - c_v\|$
- 6: walk the shortest path from v^* until cumulative TDR distance $\geq \lambda$; target $\leftarrow v_\lambda$; base cost $\leftarrow c_{\text{sub}}$
- 7: **end if**
- 8: **for** each CEM iteration **do**
- 9: sample 300 action-block sequences; $\hat{z}_{1..K} \leftarrow P(\cdot)$
- 10: $c \leftarrow \widetilde{\text{base}}(\hat{z}_K) + \widetilde{\text{ET}}(\hat{z}_K)$ with ET from equation (2)
- 11: refit the CEM Gaussian on the 30 lowest- c candidates
- 12: **end for**
- 13: execute the mean sequence for $5K$ steps; $t \leftarrow t + 5K$

5 EXPERIMENTS

5.1 SETUP

Key points this section must land.

- Fixed task pools: 200 (start, goal) pairs per protocol per environment drawn once and shared by every method and seed; a “seed” changes CEM’s draws (and, where stated, the learned assets), never the tasks. State how many of the 200 each table uses.
- Protocols: same-episode offset $K \in \{25, 50, 100, 150\}$ with budget $2K$ (cap 250); cross-episode with budget 250 (Push-T), 200 (Reacher), [TODO: Cube].
- Success is the environment’s own predicate. Say it for each environment.
- Baselines: L2 (LeWM); L2 with more search; TDR-only cost; graph cost-to-go; oracle subgoal; [TODO: TRM-style learned cost; SAGE]. Random floor.

Environments and data. Push-T (Maes et al., 2026): 18,685 expert episodes; success when agent and block position errors are below 20 px and block angle error below $\pi/9$. Reacher (dm_control): 10,000 episodes of 201 frames; success when both joints are within 0.05 rad of the goal configuration (raw difference, no angle wrap; the dataset’s shoulder angle is unwrapped over ± 8 rad). OGBench Cube (Park et al., 2025): 10,000 episodes of 201 frames; [TODO: success predicate as scored]. Frozen LeWM checkpoints are the public ones for each environment. [TODO: Two-Room.]

Protocols. For a same-episode offset K , the goal is the frame K steps after the start on the same trajectory and the budget is $2K$ steps. For the cross-episode protocol the start and goal are uniformly random frames of two different episodes with no distance or reachability filter and a budget of one full episode. We draw 200 tasks per protocol once and use the same pool for every method and every seed; unless stated, tables use the first 50 tasks and five CEM seeds (250 task-seed pairs per cell). Learned assets (TDR, graph, critic) use seed 0 unless stated; section B separates asset seeds from CEM seeds.

Baselines. L2 is LeWM’s planner. TDR scores endpoints by $\|\psi(\hat{z}_K) - \psi(z_g)\|$ with no graph; CTG uses the graph cost-to-go $\min_v \|\psi(\hat{z}_K) - c_v\| + D_g[v]$ as a terminal value with no subgoal commitment. Random takes a uniform action from the env’s own action space at every step (no model, no planning): the floor L2 and Ours must clear (table 1). [TODO: L2 with double horizon and search budget; oracle subgoal (true +25 frame); TRM-style learned pairwise cost; SAGE if code is released.]

Table 1: Success rate (%) on the fixed, held-out `task200u` pools (first 50 tasks; pre-solved pairs rejected at sampling, and all planning assets — TDR, graph, critic — prepared from training episodes only, disjoint from the task pool), mean $\pm$ sd over five CEM seeds (250 task-seed pairs per cell). Same tasks, CEM and predictor for both rows. Random is a uniform action from the env’s own action space at every step (no model, no planning): the floor L2 and Ours must clear.

Env	Method	same-ep 25	same-ep 50	same-ep 100	cross-ep
Push-T	Random	4.0 ± 1.3	0.4 ± 0.8	0.0 ± 0.0	0.4 ± 0.8
	L2 (LeWM)	91.2 ± 1.0	40.8 ± 3.7	13.2 ± 3.2	14.4 ± 3.9
	Ours	88.4 ± 2.3	70.0 ± 2.2	46.4 ± 3.2	42.4 ± 3.2
Reacher	Random	8.0 ± 3.8	8.4 ± 0.8	9.2 ± 3.2	2.8 ± 1.0
	L2 (LeWM)	77.2 ± 4.8	90.8 ± 1.0	82.8 ± 1.0	29.2 ± 2.4
	Ours	76.0 ± 4.6	98.0 ± 1.3	97.6 ± 0.8	65.6 ± 2.0
Cube	Random	21.2 ± 3.0	12.8 ± 2.0	4.8 ± 4.1	3.6 ± 2.3
	L2 (LeWM)	56.0 ± 3.8	34.4 ± 3.2	35.6 ± 1.5	16.4 ± 1.5
	Ours	62.0 ± 4.7	49.6 ± 4.6	50.4 ± 3.2	27.6 ± 6.1
Two-Room	L2 (LeWM)	[TODO:]			
	Ours	[TODO:]			

5.2 MAIN RESULT

Key points this section must land.

- One table, three (four) environments, L2 vs the full method, four protocols, mean $\pm$ sd over seeds plus pooled hits. Bold the winner only where the paired difference is significant.
- The sentence to write: same at one horizon, better at two, much better beyond, and $4\times$ / $2.6\times$ on cross-episode, every seed.

On Push-T (table 1) the method is 2.8 points under L2 at one horizon (88.4 vs 91.2), gains 29 points at two (70.0 vs 40.8), and is the strongest configuration in our study at offset 100 and on cross-episode goals (46.4 and 42.4 against 13.2 and 14.4). On Reacher L2 is strong for one or two plans and then stalls outside the 0.05-rad band (82.8 at offset 100, 29.2 cross-episode); the method reaches 97.6% at offset 100 and 65.6% cross-episode, roughly +15 and +36 points. At offset 25 on Reacher the method is 1.2 points under L2, inside CEM noise (section C). On Cube the method wins at every protocol, including offset 25 (62.0 vs 56.0) where Push-T and Reacher do not: +6.0 at offset 25, +15.2 at offset 50, +14.8 at offset 100, and +11.2 on cross-episode goals.

5.3 ABLATION LADDER

Key points this section must land.

- One table, one row per added component, Push-T, all four protocols. The reader should see that (a) the subgoal fixes offset 50, (b) the switch closes offset 25 and lifts the rest, (c) the critic is what moves cross-episode, (d) ET beats $-\log V$ at every β .
- Controls: critic without graph (12–18% cross), graph without critic (18–20%), metric without graph (TDR, CTG), waypoint tracking and direction costs (worse than L2).
- What did not work belongs here too: faster replanning, retrieval warm-start, coarser graphs.

The subgoal fixes direction. Replacing the terminal cost with the distance to the graph node one horizon along the route lifts offset 50 from 42.0 to 51.6% and offset 100 from 13.2 to 22.4%, and the graph is doing it, not the metric: over five seeds the TDR cost alone gives 48.4% at offset 50, the cost-to-go 47.2%, the subgoal 51.6% (table 2). The mechanism is visible in the final error: L2’s closest approach at offset 50 is 32 px but it ends at 76 px, whereas the subgoal objective’s closest approach and final error are both 19 px. A target inside the range where the cost still ranks correctly keeps the planner converging instead of wandering. Waypoint tracking over the five intermediate predicted latents (42%) and a direction-only progress term (26%) are both worse than L2 at offset 50.

Table 2: Push-T ablation ladder on the fixed `task200u` tasks (50 tasks, five CEM seeds, mean $\pm$ sd). Each block adds one component. “old” uses $-\log V$ with per-population standardisation; ET is the budget-capped expected-hitting-time critic. $H_{TD} = 4$ ’s graph is disconnected (240–278 components vs. 1 for $H_{TD} \in \{8, 12, 16\}$), which likely explains its lower same-ep 25 number. Full per-seed tables in section A.

configuration	same-ep 25	same-ep 50	same-ep 100	cross-ep
L2 (LeWM)	91.2 $\pm$ 1.0	42.0 $\pm$ 4.6	13.2 $\pm$ 3.7	14.0 $\pm$ 4.6
TDR terminal cost (no graph)	83.2 $\pm$ 4.1	48.4 $\pm$ 1.5	22.4 $\pm$ 5.3	14.0 $\pm$ 1.3
graph cost-to-go (no subgoal)	83.2 $\pm$ 4.1	47.2 $\pm$ 1.6	21.6 $\pm$ 4.6	14.0 $\pm$ 1.3
graph subgoal, lookahead u_{25}	80.8 $\pm$ 2.7	51.6 $\pm$ 2.3	22.4 $\pm$ 5.0	15.2 $\pm$ 3.0
+ ET critic, $\beta = 1$ (no switch)	86.0 $\pm$ 3.4	64.8 $\pm$ 3.0	43.6 $\pm$ 5.0	35.2 $\pm$ 2.0
+ final switch at λ , L2 in final phase (A)	88.0 $\pm$ 2.5	62.4 $\pm$ 2.7	30.8 $\pm$ 6.5	17.2 $\pm$ 2.7
A + critic $-\log V$, $\beta = 1$ (old)	77.6 $\pm$ 1.5	69.2 $\pm$ 2.4	47.2 $\pm$ 5.9	34.4 $\pm$ 7.3
A + ET critic, $\beta = 0.5$	88.4 $\pm$ 2.0	69.2 $\pm$ 2.7	43.2 $\pm$ 3.7	28.0 $\pm$ 3.8
A + ET critic, $\beta = 1$ (ours)	87.6 $\pm$ 2.3	67.6 $\pm$ 1.5	47.2 $\pm$ 6.9	40.0 $\pm$ 1.3
A + ET critic, $\beta = 2$	87.6 $\pm$ 3.2	72.4 $\pm$ 4.1	42.8 $\pm$ 4.1	35.2 $\pm$ 3.5
A + ET critic, $\beta = 4$	86.8 $\pm$ 2.0	65.6 $\pm$ 2.3	48.4 $\pm$ 5.0	38.4 $\pm$ 3.4
L2 + ET critic, $\beta = 1$ (no graph)	89.2 $\pm$ 3.0	58.4 $\pm$ 5.1	38.4 $\pm$ 7.1	31.6 $\pm$ 2.3
graph subgoal (non-TDR) + final switch + ET critic, $\beta = 1$	86.0 $\pm$ 2.2	63.6 $\pm$ 4.8	45.2 $\pm$ 3.5	36.4 $\pm$ 2.9
A + ET critic, $\beta = 1$, $H_{TD} = 4$	82.8 $\pm$ 2.0	64.0 $\pm$ 1.3	48.4 $\pm$ 6.1	35.2 $\pm$ 2.7
A + ET critic, $\beta = 1$, $H_{TD} = 12$	89.6 $\pm$ 2.0	68.0 $\pm$ 3.8	48.0 $\pm$ 4.0	32.4 $\pm$ 6.1
A + ET critic, $\beta = 1$, $H_{TD} = 16$	91.2 $\pm$ 2.4	64.4 $\pm$ 6.6	47.6 $\pm$ 3.4	36.8 $\pm$ 4.5

The switch fixes resolution. Handing the final horizon back to c_{L2} once the goal is one lookahead away closes the offset-25 gap (86.8 $\rightarrow$ 90.8, L2 91.2) and also lifts every other protocol (57.2 $\rightarrow$ 61.2, 21.6 $\rightarrow$ 28.8): the medoid-radius floor was costing successes everywhere, not just at short range.

The critic fixes reachability, and only in composition. Cross-episode does not move with the graph alone (18–20%) nor with the critic alone on top of L2 (12–18% on seed 0); together they reach 31–44%. The critic without the graph still fights L2’s flat long-range ranking; the graph without the critic knows the route but lets the block drift once the first approach is made (median final error 125 px vs 26 px with the critic).

Expected hitting time removes the β dependence. With $-\log V$, β trades protocols: 0.5 is best at offset 25, 2 on cross-episode, and every value loses 8–22 points at offset 25 because the critic saturates there (67% of the CEM population is feasible at offset 25; 39–41% at offset 100 and cross-episode) and per-population standardisation stretches its calibration noise to a full vote. With ET, $\beta = 1$ is at least as good as $\beta = 2$ on every protocol and the row is best or tied-best everywhere. Two β -free alternatives confirm the reading: the hard feasibility filter ($\tau = 0.5$) is tied-best at offset 50 but ten points behind on cross-episode, because a threshold discards the ranking among infeasible candidates that ET keeps; converting both terms to calibrated step units and adding them without standardisation works but dilutes the veto on far pairs (32.0 cross-episode).

What did not help. Replanning every 5 or 10 steps hurts offset 50 (58 $\rightarrow$ 36%) and only slightly helps cross-episode: node re-selection flips targets before a 25-step plan has done its work. Warm-starting CEM from the logged 25-action block of the nearest dataset frame executes the first pushes better but does not stop the drift, and adds nothing once the critic is present. A coarser graph ($H_{TD} = 12$ or 16) and a stricter temporal-efficiency filter ($\theta_{TE} = 0.99$) are flat to worse.

Table 3: Per-pair diagnostic, Push-T cross-episode, seed 0, 50 pairs. D_g : graph geodesic from start to goal in environment steps via the TDR calibration. “near”: initial block error < 60 px and $< 30^\circ$. Cells are successes / pairs in the bin. [TODO: Extend to all seeds; add the ET row.]

objective	$D_g \in [0, 50)$ (14)	$D_g \in [50, 100)$ (33)	$D_g \geq 100$ (3)	near (8)	far (42)
L2	2	2	0	1	3
graph subgoal	3	2	0	2	3
L2 + critic, $\beta = 2$	3	6	0	1	8
graph subgoal + critic, $\beta = 1$	3	9	1	3	10
graph subgoal + critic, $\beta = 2$	7	12	0	6	13
graph subgoal + critic, $\beta = 4$	6	14	2	5	17

5.4 MECHANISTIC ANALYSIS

Key points this section must land.

- The per-pair geodesic table is the argument: the graph rates every cross-episode pair 45–120 steps away (median 66), inside a 250-step budget, and the successes concentrate in the 50–100-step bin only once the critic is present.
- Wins are not the easy pairs: “block far” pairs go from 3/42 (L2) to 17/42.
- Final error and first-hit step per method: the critic stops drift (163→26 px, $71^\circ \rightarrow 9^\circ$); ET keeps a progress gradient that $-\log V$ lacks (median first hit 95 vs 188 steps).
- Where the critic is worse than L2 (oracle Spearman 0.60 vs 0.72 at $h = 25$) is exactly where the switch removes it.

The objective knows a route; the executor needs a veto. By the graph’s own geodesic the 50 cross-episode pairs are 45–86 steps apart (median 66, max 119): all inside the budget and none beyond the range the subgoal objective handles at offset 50. Yet the subgoal objective alone succeeds on 5 of them, the same as L2. Twenty-five open-loop actions drawn from a Gaussian proposal can follow the route’s direction but not string together the pushes a 66-step expert route needs, and once the block is displaced the planner has no signal that the goal has become unreachable from where it is heading. The critic supplies exactly that signal. Table 3 shows where the extra successes come from: the 50–100-step bin (2→9–14 of 33) and the “block far” pairs (3→10–17 of 42), not the near pairs that any method solves. Median final agent-plus-block error falls from 163 px (L2) to 26 px and block angle error from 71° to 9° .

Feasibility has no progress gradient; hitting time does. Under $-\log V$ the median first-hit step rises from 96 to 248 as β goes from 1 to 4: a more cautious planner is a slower one, and at $\beta = 8$ one seed has no success before step 175. ET orders feasible endpoints by how soon they arrive, and the headline configuration hits at a median of 95 steps on cross-episode against 188 for the best $-\log V$ row.

Where the critic is worse than L2, the switch removes it. The privileged audit ranks the critic below L2 at $h = 25$ (Spearman 0.60 vs 0.72) and far above it at $h = 50$ (0.70 vs 0.36). The final switch means the critic only votes when the goal is more than one horizon away, which is the regime where it is better. [TODO: Figure: oracle Spearman of L2 / TDR / critic vs offset.]

Predicted versus realized cost. [TODO: Repeat the single-pass predicted-vs-realized audit (section 3) for the full method: does the graph subgoal reduce the optimism gap (it should, since the target is one horizon away), and does the composed cost separate successes from failures where c_{L2} did not?]

Compute. [TODO: Table: one-off preparation (encode / TDR / graph / critic, GPU-minutes) and per-decision planning time relative to L2. The Dijkstra pass is once per goal; the criterion adds a 32-d MLP and a five-member critic per candidate.]

6 LIMITATIONS

Key points this section must land.

- Say the four things a reviewer will say first, in our words: privileged critic labels; goal frames are in the graph; one frozen model family; SAGE is stronger on same-episode long offsets.
- Each with the mitigation that exists or the experiment that would settle it.

The critic’s hindsight labels and cross-episode negatives use the dataset’s state column, so the pipeline is not pixel-only; a label-free variant trained on graph hitting times is weaker (section E) and closing that gap is open. Evaluation start and goal frames are dataset frames and the graph is built on 98% of episodes, so goals are almost always graph members; **[TODO: held-out-episode tasks]** measure the cost of that. All results use one frozen model family; whether the same graph and critic help DINO-WM or PLDM is untested. SAGE (Cheng et al., 2026) reports higher same-episode success than ours at long offsets with the same frozen model by training a subgoal generator on 400k expert windows; we do not claim the same-episode state of the art, and we expect a generator trained on single-trajectory windows to be unable to stitch, which the cross-episode protocol would show **[TODO: run it]**. Reacher’s short-range loss (six points at offset 25) is inside CEM noise over ten seeds but consistent in sign. All data are expert demonstrations; the suboptimal-data regime where graph stitching is usually studied is not covered.

7 CONCLUSION

A frozen pixel world model’s planner fails past one horizon because its terminal cost cannot tell which candidates can still reach the goal. Giving it a data-supported target one horizon along a shortest path over the dataset, a budget-aware hitting-time estimate of what remains, and its own objective for the last horizon repairs mid-range, long-range and cross-episode goal reaching with nothing in the model or the planner retrained. The three pieces fix three different failures, and the per-pair diagnostics say why the last one is necessary: the graph knows a route for every cross-episode pair, and the planner cannot execute it without a reachability term.

AI USE STATEMENT

In this work, we used generative AI tools (an LLM-based coding assistant) to write and refactor implementation and analysis code, to help with literature search, and to edit text. We have not used generative AI tools to generate synthetic data, to develop the conceptual framework, or to formulate mathematical claims. We have reviewed all AI-assisted work: LLM-written code was read and tested by the authors, every reported number was produced by runs the authors launched and checked, and citations found with AI assistance were verified against the original papers. We take responsibility for the final content of this work, including text, claims or artifacts produced with the aid of generative AI. **[TODO: Confirm the disclosure list with all authors.]**

ETHICS STATEMENT

This work uses simulated benchmarks with public datasets and checkpoints and involves no human subjects or sensitive data.

REPRODUCIBILITY STATEMENT

[TODO: Point to the anonymised code, the fixed task-pool files, the seeds, the TDR / graph / critic checkpoints, and the exact CEM configuration. Mention the MuJoCo render-version check (section D) and the action-normalisation convention (predictor inputs are z-scored with the full dataset’s action statistics).]

REFERENCES

ALPS authors. Laplacian representations for decision-time planning. In *International Conference on Machine Learning (ICML)*, 2026. verify authors; code at github.com/machado-research/ALPS.

- Mahmoud Assran et al. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025. verify.
- Seungho Baek, Taegeon Park, Jongchan Park, Seungjun Oh, and Yusung Kim. Graph-assisted stitching for offline hierarchical reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2025. arXiv:2506.07744.
- Jiaxin Bai and Jiaxuan Xiong. Temporal-distance JEPA: Plan-aware representation learning for latent world model predictive control. *arXiv preprint arXiv:2607.25337*, 2026.
- Caselli, Massafra, Punzo, Lo Sardo, Pantelidis, and Bhethanabhotla. Mind the gap: Promises and pitfalls of hierarchical planning in LeWorldModel. *arXiv preprint arXiv:2607.12547*, 2026. verify author first names.
- Letian Cheng, Qi Zhang, and Yisen Wang. SAGE: Subgoal-conditioned action generation for latent world model planning. *arXiv preprint arXiv:2607.17973*, 2026.
- Matthieu Destrade, Oumayma Bounou, Quentin Le Lidec, Jean Ponce, and Yann LeCun. Value-guided action planning with JEPA world models. *arXiv preprint arXiv:2601.00844*, 2026. World Modeling Workshop 2026.
- Scott Emmons, Ajay Jain, Michael Laskin, Thanard Kurutach, Pieter Abbeel, and Deepak Pathak. Sparse graphical memory for robust planning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- Benjamin Eysenbach, Ruslan Salakhutdinov, and Sergey Levine. Search on the replay buffer: Bridging planning and reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- GC terminal value authors. Goal-conditioned terminal value estimation for real-time and multi-task model predictive control. *arXiv preprint*, 2024. verify: locate the arXiv id.
- Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *International Conference on Machine Learning (ICML)*, 2019.
- Danijar Hafner, Kuang-Huei Lee, Ian Fischer, and Pieter Abbeel. Deep hierarchical planning from pixels. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
- Nicklas Hansen, Xiaolong Wang, and Hao Su. Temporal difference learning for model predictive control. In *International Conference on Machine Learning (ICML)*, 2022.
- Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations (ICLR)*, 2024.
- Jiaming Hu, Yan Zheng, and Tian Wang. SCALE: State-calibrated latent embeddings for JEPA planning in the right geometry. *arXiv preprint arXiv:2608.16287*, 2026.
- Liangyu Li, Shengzhi Wang, Libin Qiu, Mingliang Xiong, and Qingwen Liu. Beyond Euclidean proximity: Repairing latent world models with horizon-matched trajectory reachability metrics. *arXiv preprint arXiv:2605.22164*, 2026a.
- Wenyuan Li, Guang Li, Keisuke Maeda, Takahiro Ogawa, and Miki Haseyama. Predictive but not plannable: RC-aux for latent world models. *arXiv preprint arXiv:2605.07278*, 2026b.
- Kendall Lowrey, Aravind Rajeswaran, Sham Kakade, Emanuel Todorov, and Igor Mordatch. Plan online, learn offline: Efficient learning and exploration via model-based control. In *International Conference on Learning Representations (ICLR)*, 2019.
- Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorld-Model: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026.

- Sergi Masip, Jonathan Swinnen, Yutong Hu, Renaud Detry, and Tinne Tuytelaars. FF-JEPA: Long-horizon planning in world models with latent planners. *arXiv preprint arXiv:2606.09311*, 2026.
- Evgenii Opryshko, Junwei Quan, Claas Voelcker, Yilun Du, and Igor Gilitschenski. Test-time graph search for goal-conditioned reinforcement learning. *arXiv preprint arXiv:2510.07257*, 2025.
- Seohong Park, Tobias Kreiman, and Sergey Levine. Foundation policies with Hilbert representations. In *International Conference on Machine Learning (ICML)*, 2024.
- Seohong Park, Kevin Frans, Benjamin Eysenbach, and Sergey Levine. OGBench: Benchmarking offline goal-conditioned RL. In *International Conference on Learning Representations (ICLR)*, 2025.
- Nikolay Savinov, Alexey Dosovitskiy, and Vladlen Koltun. Semi-parametric topological memory for navigation. In *International Conference on Learning Representations (ICLR)*, 2018.
- Vlad Sobal, Wancong Zhang, Kynghyun Cho, Randall Balestriero, Tim G. J. Rodriguez, and Yann LeCun. Learning from reward-free offline data: A case for planning with latent dynamics models. *arXiv preprint arXiv:2502.14819*, 2025. verify author list.
- stable-worldmodel authors. stable-worldmodel: A platform for reproducible world modeling research and evaluation. *arXiv preprint arXiv:2605.21800*, 2026. verify authors.
- Temporal straightening authors. Temporal straightening for latent planning. *arXiv preprint arXiv:2603.12231*, 2026. verify authors.
- Basile Terver, Tsung-Yen Yang, Jean Ponce, Adrien Bardes, and Yann LeCun. What drives success in physical planning with joint-embedding predictive world models? *arXiv preprint arXiv:2512.24497*, 2025.
- Jiawei Wang, Ke Rui, Yushen Zuo, Yichun Feng, and Minglei Li. Decision-metric alignment in latent world models: Diagnostics and action-conditioned objectives for MPC planning. *arXiv preprint arXiv:2608.18746*, 2026.
- Tongzhou Wang, Antonio Torralba, Phillip Isola, and Amy Zhang. Optimal goal-reaching reinforcement learning via quasimetric learning. In *International Conference on Machine Learning (ICML)*, 2023.
- Wancong Zhang, Basile Terver, Artem Zhohus, Soham Chitnis, Harsh Sutaria, Mido Assran, Randall Balestriero, Amir Bar, Adrien Bardes, Yann LeCun, and Nicolas Ballas. Hierarchical planning with latent world models. *arXiv preprint arXiv:2604.03208*, 2026.
- Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. *arXiv preprint arXiv:2411.04983*, 2024.

A PER-SEED RESULTS

[TODO: Paste the per-seed blocks from docs/gas-mpc/results_table.tex and results_table_reacher.tex; add Cube. Report the “solved in ≥ 1 seed / every seed” columns.]

B ASSET SEEDS VERSUS CEM SEEDS

[TODO: Retrain TDR / graph / critic on three seeds and report the variance that comes from the learned assets separately from CEM sampling.]

C REACHER DETAILS AND THE SHORT-RANGE GAP

[TODO: Ten-seed same-25 comparison (82.6 vs 85.2, $p = 0.37$); the critic_final=false variant; calibration table ($H_{TD} = 3.93$, $\lambda = 5.78$ TDR units); graph statistics (535 nodes, 1.9k edges); critic diagnostics (hindsight AUROC 0.906, ECE 0.010).]

D LIVE RENDERING MUST MATCH THE DATASET

The frozen encoder is sensitive to renderer versions. Reacher frames rendered with MuJoCo 3.12 lack the floor checker texture of the dataset’s MuJoCo 3.5 frames; the pixel difference is 18/255 on average and moves the latent by a median of 15 units, about one 25-step gap, so the terminal cost never reaches zero even with the arm at the goal. This alone took LeWM’s own planner from 87% to 65% at offset 25. Our launcher refuses to evaluate unless three dataset frames re-render to within 4/255 of the stored pixels; Push-T (pygame) and Cube pass with 0.5/255 and $\leq 0.2/255$.

E A PIXEL-ONLY CRITIC

[TODO: Graph hitting-time labels instead of privileged hindsight labels: oracle ranking and closed-loop numbers of the label-free critic; the gap to the privileged critic.]

F PREDICTOR ROLLOUT ERROR BY HORIZON

Rolling the frozen predictor forward under real logged actions and comparing to the true encoded endpoint, Push-T’s latent rollout error grows with horizon but stays well below the scale of two unrelated frames (mean 19.6) even at $H = 100$ (table 4 and figure 2): the predictor degrades gracefully rather than collapsing to an uninformative guess. This matches theory: section G’s isotropic-Gaussian fit to the latent cloud ($d = 192$, empirical $\|z - \mu\|$ mean 13.88) predicts a random-pair distance of $\sqrt{2} \times 13.88 \approx 19.63$, since $z_1 - z_2 \sim \mathcal{N}(0, 2\sigma^2 I_d)$ for two independent draws – essentially exact agreement with the measured 19.6.

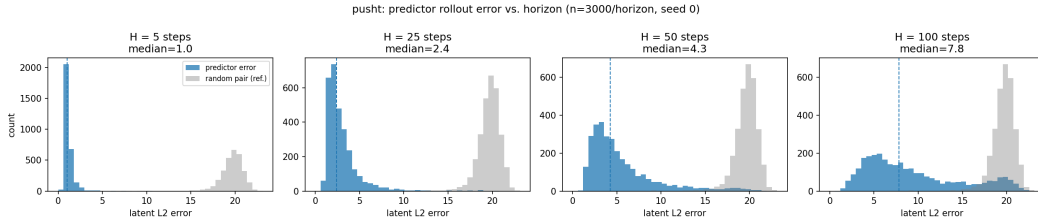


Figure 2: Push-T predictor latent rollout error (blue) vs. horizon, against the random-pair reference distribution (gray).

Task difficulty: how far apart are the actual start/goal pairs? The random-pair reference above (mean 19.6 on the 3,000-sample draw used for figure 2) is the scale at which two frames carry no information about each other. Sampling same-episode (start, start+ H) pairs the same way figure 2 samples its rollout starts, but reading off the true encoded distance $\|z_{\text{start}} - z_{\text{start}+H}\|$ instead of rolling the predictor forward (table 4), shows $H = 5$ pairs sit far below the random-pair scale (median 5.3, std 2.8) while $H = 25$ is already most of the way there (17.1) and $H = 50/H = 100$ are close to indistinguishable from it (19.4, 19.6) – consistent with section 3’s Spearman collapse past $H=50$: an arbitrarily sampled goal at that range is close to latent-distance-indistinguishable from a random unrelated frame, exactly the regime where a cost that only reads terminal z -space L2 has nothing left to discriminate on. Relative to that task scale, the predictor’s own rollout error is already 18.9% of the gap at $H = 5$ and grows to 39.8% by $H = 100$ – a sizeable and growing fraction of the very distance CEM is trying to cross, consistent with table 1’s OUR-vs-L2 gap widening at longer horizons. The task200u pool used for evaluation agrees closely with this same-episode sampling wherever the two overlap (same-ep 25/50/100 medians 17.2/19.6/19.5, std 4.1/3.7/2.1), so the two measurements of task difficulty are not an artifact of either sampling scheme.

Table 4: Push-T: predictor latent rollout error vs. the true same-episode start→goal distance at the same horizon (both $n = 3,000/\text{horizon}$, identically sampled), median $\pm$ std in latent L2. Random-pair reference: mean 19.6 (theory: $\sqrt{2} \times 13.88 \approx 19.63$, section G). Relative error is the predictor-error median divided by the start→goal-distance median.

	$H = 5$	25	50	100
predictor error	1.00 ± 0.56	2.41 ± 2.09	4.28 ± 3.82	7.80 ± 5.35
start→goal distance	5.29 ± 2.76	17.06 ± 4.19	19.45 ± 3.29	19.60 ± 2.43
relative error (%)	18.9	14.1	22.0	39.8

Scripts: `scripts/task.start.goal.distance.py` (task200u pool, per protocol) and `scripts/direct.horizon.distance.py` (same-episode sampling at a fixed horizon, reusing predictor.error.by.horizon.py’s own sampler).

This still leaves the planning-side question: does the L2 baseline do better with frequent short replans or fewer wide ones? Same-ep 25’s budget (2×25) already gives the default cadence a $2 \times H$ structure at $H = 25$; table 5 extends it to larger H by widening the chunk to match the offset.

Table 5: Push-T L2 baseline: default 25-step-chunk cadence vs. chunks widened to the offset ($2 \times H$, or $3 \times H$ for cross-ep). Success (%), mean $\pm$ sd over 5 CEM seeds.

protocol	chunking	mean $\pm$ sd
same-ep 25	default = 2×25	91.2 ± 1.1
same-ep 50	default (4×25) $2 \times H$ (2×50)	40.8 ± 4.1 34.4 ± 3.0
same-ep 100	default (8×25) $2 \times H$ (2×100)	13.2 ± 3.6 5.6 ± 2.5
cross-ep	default (10×25) $3 \times H$ (3×100)	14.4 ± 4.3 4.8 ± 2.2

Same-ep 25’s 91.2% sits where the two cadences coincide. Past $H = 25$, widening the chunk toward $2H$ costs the L2 baseline 6–10 points, growing with how much of the budget one chunk covers – consistent with predictor error compounding across an open-loop commitment, and confirming the default 25-step cadence used elsewhere in this paper is the more favorable setting for L2, not an artifact inflating the OUR-vs-L2 gaps in table 1.

G WHY A STRAIGHT LATENT LINE GOES OUT OF DISTRIBUTION

Section F shows the predictor itself degrades gracefully with horizon; this section asks why the terminal L2 objective built on top of it still fails past $H \approx 50$ (oracle Spearman $0.72 \rightarrow 0.36$, section 3).

In a high-dimensional isotropic Gaussian, essentially all probability mass concentrates on a thin shell at radius $\sigma\sqrt{d}$ rather than near the mean, because the volume element $r^{d-1}dr$ grows faster than the Gaussian density falls off until $r \approx \sigma\sqrt{d}$.¹ If LeWM’s Push-T latent cloud approximates such a shell, a straight line drawn between two points *on* the shell passes through the ball’s interior, which the shell distribution assigns almost no mass – exactly the regime a z -space-L2 CEM plan implicitly cuts through when it scores a candidate purely by terminal distance to a goal many steps away.

The latent cloud is close to an isotropic shell. Fitting an isotropic Gaussian $\mathcal{N}(\mu, \sigma^2 I_d)$ to all 2,336,736 encoded Push-T frames ($d = 192$, μ and $\sigma^2 = \text{tr}(\Sigma)/d$ from the empirical mean and covariance) predicts $\|z - \mu\|$ should follow a $\chi_d(\sigma)$ distribution with mean 13.88. The true encoded frames match this almost exactly (empirical mean 13.88, figure 3), and the participation ratio $\text{tr}(\Sigma)^2/\text{tr}(\Sigma^2) = 188.5$ out of 192 dimensions (98%) confirms the covariance itself is close to isotropic, so the χ_d comparison is not being flattered by a degenerate few dominant directions.

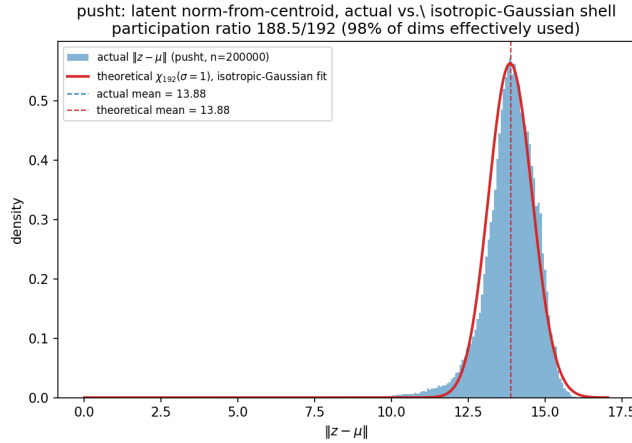


Figure 3: Push-T: actual $\|z - \mu\|$ over all 2.3M encoded frames vs. the χ_{192} prediction from an isotropic Gaussian fit to the same mean/covariance.

A straight-line midpoint is far off the shell; the real trajectory is not. For 200,000 same-episode frame pairs 100 steps apart (the same-ep-100 horizon), both endpoints sit on the shell as expected (start mean 14.13, goal mean 13.56), and so does the actual logged frame at the trajectory’s true midpoint (mean 13.81). The linear interpolation $\frac{1}{2}(z_{\text{start}} + z_{\text{goal}})$ does not: its mean radius is 9.99, 5.48 shell- σ below the shell mean, and 96.6% of the 200,000 interpolated midpoints fall below the shell’s own 1st percentile radius (figure 4). The two populations do not overlap. This isolates the effect to interpolation itself, not to trajectory midpoints being unusual in general: the real dynamics stay on the shell, curving around it rather than cutting through its interior, which is exactly the structure a graph built in a temporal-distance representation is meant to follow instead of a raw latent straight line (sections 4 and 5.2).

Scope. This diagnostic is Push-T-only and tests the naive linear interpolation, not the predictor’s own imagined rollout under the z -space-L2 objective CEM actually optimizes; whether the predictor’s rollout exhibits the same off-shell dip, and whether the effect holds on Reacher and Cube, is not yet measured. *Scripts:* `scripts/gas_mpc-gaussian-shell-diag.py` and `scripts/gas_mpc-gaussian-shell-interp-diag.py`.

H CANDIDATE RANKING WITHIN CEM’S OWN ELITES

A separate check of whether OUR’s cost ranks candidates better than z -space L2 not on a fixed diagnostic pool but on CEM’s own live search: for 25 held-out Push-T tasks per same-ep protocol

¹<https://aseemrb.me/blog/high-dimensional-gaussians-on-a-sphere/>

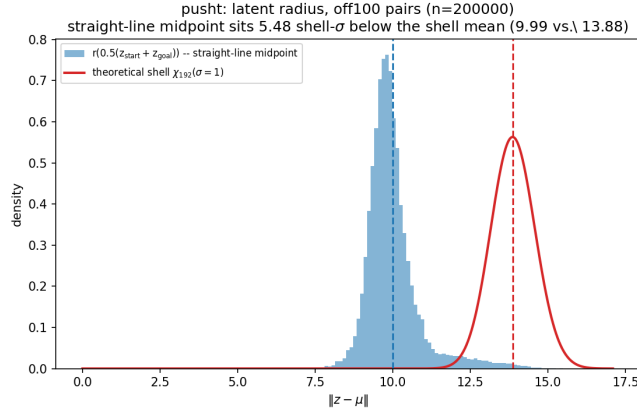


Figure 4: Push-T, 200,000 same-episode pairs 100 steps apart: radius of the straight-line midpoint $\frac{1}{2}(z_{\text{start}} + z_{\text{goal}})$ vs. the shell χ_{192} reference. The two distributions do not overlap.

(seed 0), OUR’s first CEM solve is run to iteration 1 only – the one point where both costs score the identical, as-yet-unbiased 300-candidate population sampled from CEM’s default zero-mean initialisation, before either cost’s own elite selection has shaped anything. OUR’s 30 elite candidates at that iteration are executed for real in the simulator and scored against the privileged real-physics recovery-time oracle (sections 3 and F’s mechanism).

Table 6: Spearman(cost, oracle recovery time) over OUR’s own 30 elite candidates at CEM iteration 1, 25 tasks. Same25/50: 25/24 tasks with non-degenerate oracle labels; same100: 11/25 (the rest were fully censored even at a widened oracle cap).

protocol	$\rho(\text{OUR})$	$\rho(\text{L2})$
same-ep 25	0.327	0.396
same-ep 50	0.203	0.086
same-ep 100	0.191	−0.064

At same-ep 25, L2 is nominally ahead, though 95% task-bootstrap confidence intervals overlap. At same-ep 50 and 100, OUR is ahead, and is the only cost of the two whose interval excludes zero at both – L2’s ranking is statistically indistinguishable from chance there, while OUR’s is not. This is the same short-range/long-range crossover table 1 shows in closed-loop success, now shown at the level of a single CEM iteration’s candidate ranking rather than a full episode. *Script: scripts/gas_mpc_cem_trace_audit.py.*

I HYPERPARAMETERS

[TODO: TDR: 3×512 LayerNorm/GELU MLP, 32-d output, expectile 0.99, $\gamma = 0.99$, 50k steps, batch 1024. Graph: $\theta_{\text{TE}} = 0.9$, $H_{\text{TD}} = u_{12}$, clustering radius $H_{\text{TD}}/2$, goal attachment $\max(H_{\text{TD}}, 1.2d_{\min})$. Critic: five members, $h_{\max} = 50$, grid step 5, 60k steps, Bellman + monotonicity, cross-episode negative gate (Push-T ≥ 100 px block pose; Reacher ≥ 2.5 rad; Cube ≥ 0.48 m). CEM: 300 / 30 / 30, $K = 5$, open-loop 25 steps. $\beta = 1$.]

J SUCCESS RATE UNDER A LOOSENED SUCCESS THRESHOLD

Table 1’s success predicate requires combined agent+block position error < 20 px and block angle error $< 20^\circ$ simultaneously. Table 7 recomputes success directly from the saved state trajectories of the same task200u eval runs under two loosened joint thresholds, for OUR and the L2 baseline, on the two longer-horizon protocols where the gap is largest.

Table 7: Push-T success (%) under loosened joint position/angle thresholds, same-ep 100 and cross-ep, 50 tasks $\times$ 5 CEM seeds. Recomputed from the task200u eval trajectories underlying table 1; the 20 px/20° row reproduces table 1 exactly.

protocol	threshold	OUR	L2
same-ep 100	20 px / 20° (default)	46.4	13.2
	25 px / 25°	52.8	14.8
	30 px / 30°	58.0	16.0
cross-ep	20 px / 20° (default)	42.4	14.4
	25 px / 25°	51.2	19.2
	30 px / 30°	57.6	22.8

Both methods gain 6–15 points as the threshold loosens, consistent with most failures being near-misses rather than gross misses. The OUR-vs-L2 gap widens rather than shrinks under looser thresholds (same-ep 100: +33.2 $\rightarrow$ +42.0; cross-ep: +28.0 $\rightarrow$ +34.8), so OUR’s advantage in table 1 is not an artifact of the strict threshold disproportionately penalizing OUR’s near-misses – if anything, L2’s failures sit further from the success boundary.

Success rate by budget. Table 8 recomputes success in successive 50-step env-step windows on the same task200u eval runs, at the default 20 px/20° threshold, for same-ep 100 and cross-ep. Each window reports the cumulative success rate through that budget, with the bracketed value giving a hazard rate: of the tasks still unsolved entering the window, what fraction were solved within it.

Table 8: Push-T success (%) by env-step budget window, same-ep 100 and cross-ep, 50 tasks $\times$ 5 CEM seeds, default 20 px/20° threshold. Each cell is the cumulative success rate through that budget; bracketed is the hazard rate – % of tasks still unsolved entering the window that were solved within it.

window (env steps)	same-ep 100		cross-ep	
	OUR	L2	OUR	L2
0–50	8.8 (8.8%)	8.8 (8.8%)	14.8 (14.8%)	6.0 (6.0%)
50–100	23.2 (15.8%)	12.0 (3.5%)	30.4 (18.3%)	12.0 (6.4%)
100–150	36.4 (17.2%)	12.4 (0.5%)	36.4 (8.6%)	13.2 (1.4%)
150–200	46.4 (15.7%)	13.2 (0.9%)	38.8 (3.8%)	14.0 (0.9%)
200–250	–	–	42.4 (5.9%)	14.4 (0.5%)

L2’s hazard rate collapses almost immediately – 8.8% in the first window, then under 3.5% in every window after, so additional replans buy it almost nothing past budget 50. OUR’s hazard rate stays in the 8.6–18.3% range through same-ep 100’s full budget and cross-ep’s first three windows, meaning a meaningful fraction of the tasks OUR hasn’t yet solved keep getting solved by later replans, not just the easy fraction up front. This is where the bulk of OUR’s advantage over L2 in table 1 accumulates: not primarily an early edge, but a sustained one that L2’s open-loop terminal-L2 objective cannot keep converting into progress.